Table of Contents

🚀 Week 8 Hands-On Guide: Build a Student Management System with JDBC

Step-by-Step Tutorial for Students — Using YourWeek 7 `university_db`

📋 Before You Start — Checklist

📁 PART A — Set Up the Project in IntelliJ IDEA

Step 1 — Create a New Module Inside Your Existing Course Project

Step 2 — Add the PostgreSQL JDBC Library to the Module

Step 3 — Create the Package and Source Files

Step 4 — Verify YourWeek 7 Database in pgAdmin

💻 PART B — Write the Code (File by File)

FILE 1 of 5 — `DatabaseConfig.java`

✅ Test File 1 Before Continuing

FILE 2 of 5 — `Student.java`

FILE 3 of 5 — `StudentDAO.java`

FILE 4 of 5 — `StudentDAOImpl.java`

✅ Quick Test for File 4

FILE 5 of 5 — `StudentManagementApp.java`

🏃 PART C — Run the Application

Step 1 — Launch

Step 2 — Test Every Menu Option

🔍 PART D — How Everything Connects

⚠️ Common Mistakes to Avoid

Mistake 1 — Using Statement instead of PreparedStatement (SQL Injection!)

Mistake 2 — Wrong parameter index (most common beginner error!)

Mistake 3 — Wrong execute method

Mistake 4 — Forgetting to close resources

Mistake 5 — Missing WHERE clause on UPDATE or DELETE

Mistake 6 — Wrong column name in ResultSet (specific toWeek 7 schema)

🏆 Extension Challenges (Homework)

Challenge 1 — Search Students by Name (LIKE)

Challenge 2 — View a Student's Enrolled Courses

Challenge 3 — Department Summary Report

Challenge 4 — Enroll a Student in a Course (Transaction)

DAO Methods → SQL Mapping

JDBC Rules — Always Remember

🚀 Week 8 Hands-On Guide: Build a Student Management System with JDBC

Step-by-Step Tutorial for Students — Using YourWeek 7 `university_db`

> ***What you will build:*** *A fully working console application in Java that connects to the* ***same*** `university_db` ***database you built last week*** *— with its full schema of* `departments`, `students`, `teachers`, `courses`, *and* `enrollments` *— and lets you Add, View, Search, Update, and Delete students using professional software architecture (DAO pattern).*

📋 Before You Start — Checklist

Make sure you have all of these ready before writing a single line of Java:

Requirement	Status
✅ IntelliJ IDEA (Community or Ultimate)	https://www.jetbrains.com/idea/download/
✅ Java JDK 17 or higher	https://adoptium.net/
✅ PostgreSQL installed and running	https://www.postgresql.org/download/
✅ pgAdmin 4	Installed automatically with PostgreSQL
✅ `university_db` with allWeek 7 tables	`departments`, `students`, `teachers`, `courses`, `enrollments`

> 💡 ***Don't have theWeek 7 data yet?*** *Jump to* ***Part A — Step 4*** *to run the full schema and sample data SQL before touching any Java code.*

📁 PART A — Set Up the Project in IntelliJ IDEA

Step 1 — Create a New Module Inside Your Existing Course Project

> 💡 ***Why a Module?***
> *In class we keep all weekly exercises inside **one IntelliJ project** (e.g. `JavaCourse`). Each week's work lives in its own **module** inside that project. This keeps everything organised and lets you share settings like the JDK across all weeks.*

If you already have a course project open (most common):

1. Open your existing course project in IntelliJ IDEA (e.g. `JavaCourse`)
2. In the top menu go to **File** → **New** → **Module...**
3. The "New Module" dialog opens — fill in the settings:

Field	Value
Name	`Week7-JDBC`
Location	Leave as-is — IntelliJ places it inside your project folder automatically
Language	Java
Build system	**IntelliJ** *(simple, no extra files needed)*
JDK	Select your installed JDK (17 or higher)

4. Click **"Create"**
5. IntelliJ adds the new module — you will see `Week7-JDBC` appear in the **Project** panel on the left, alongside your other week folders.

> 📂 ***What your project structure looks like after this step:***

```
JavaCourse/                  ← your existing course project
├── Week1-Basics/
├── Week2-OOP/
├── ...
├──Week7-SQL/
└──Week8-JDBC/              ← new module just created
    └── src/                 ← all your Java files go here
```

If you are starting a brand-new project from scratch:

1. Open IntelliJ IDEA and click **"New Project"** (or **File** → **New** → **Project...**)
2. On the left panel select **"Java"**
3. Fill in:

Field	Value
Name	`JavaCourse`
Location	Choose your preferred folder
Language	Java
Build system	**IntelliJ**
JDK	Select your installed JDK (17 or higher)

4. Click **"Create"** — this is now your course project
5. Then follow the **"already have a course project"** steps above to add `Week7-JDBC` inside it

---

Step 2 — Add the PostgreSQL JDBC Library to the Module

You need the PostgreSQL JDBC driver so Java can talk to your database.

Download the JAR file first

1. Go to **https://jdbc.postgresql.org/download/** in your browser
2. Download the latest `.jar` file — look for the filename **`postgresql-42.7.1.jar`** (or the newest version shown)
3. Save it inside your course project's `libs/` folder:

> 💡 ***Class tip:*** *Put the JAR inside the project so everyone shares the same file. You only download it once.*
>
> ```
> JavaCourse/
> ├── libs/
> │   └── postgresql-42.7.1.jar   ← save it here
> ├──Week7-SQL/
> └──Week8-JDBC/
>  └── src/
> ```

Add the JAR to theWeek8-JDBC module

1. In IntelliJ, open **File → Project Structure**
   (shortcut: `Ctrl + Alt + Shift + S` on Windows / `Cmd + ;` on Mac)
2. In the left panel, click **"Modules"**
3. In the centre list, click **"Week7-JDBC"** to select it
4. Click the **"Dependencies"** tab on the right
5. Click the **"+"** button → choose **"JARs or Directories..."**
6. Browse to `libs/postgresql-42.7.1.jar`, select it, click **OK**
7. The JAR appears in the list with a ✅ checkbox — make sure it is checked
8. Click **"Apply"** → **"OK"**

> ✅ ***How to verify it worked:***
> *In the Project panel, expand `Week7-JDBC` → look for **"External Libraries"** → you should see `postgresql-42.7.1` listed. If it's there, the driver is ready.*

What the Dependencies tab should look like:

```
Week7-JDBC — Dependencies tab
──────────────────────────────────────
  ☑  <Module source>
  ☑  JDK 17                       (your Java version)
  ☑  postgresql-42.7.1.jar        ← just added
```

Step 3 — Create the Package and Source Files

1. In the **Project** panel, expand: `Week7-JDBC` → `src`
2. Right-click `src` → **New** → **Package**
3. Type: `com.university.jdbc` → press **Enter**

Now create **5 Java files** inside this package. Right-click the package → **New** → **Java Class** for each:

#	File Name	Type	Purpose
1	`DatabaseConfig`	Class	Manages all database connections

3	`StudentDAO`	Interface	Contract listing all CRUD operations
4	`StudentDAOImpl`	Class	Full SQL implementation of every operation
5	`StudentManagementApp`	Class	Menu and user interaction

> ⚠️ ***Create all 5 files now (leave them empty), then fill in the code in order from File 1 to File 5.***

Your final structure inside the module:

```
Week7-JDBC/
└── src/
    └── com/university/jdbc/
        ├── DatabaseConfig.java
        ├── Student.java
        ├── StudentDAO.java
        ├── StudentDAOImpl.java
        └── StudentManagementApp.java
```

Step 4 — Verify YourWeek 7 Database in pgAdmin

Before writing any Java code, confirm that `university_db` exists with all the tables and data from last week.

1. Open **pgAdmin 4**
2. Connect to your PostgreSQL server
3. Navigate to: `Servers` → `PostgreSQL` → `Databases` → `university_db`
4. Right-click `university_db` → **Query Tool**
5. Run this quick check:

```sql
-- Check all tables exist
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'public'
ORDER BY table_name;

-- Should show: courses, departments, enrollments, students, teachers

-- Check student count
SELECT COUNT(*) FROM students;
-- Should show: 8
```

If your database is missing tables or data — run this full setup SQL:

> *Skip this block if your database already has all theWeek 7 tables and data.*

```sql
-- ═══════════════════════════════════════════════
--Week 7 SCHEMA – run only if tables are missing
-- ═══════════════════════════════════════════════

-- Departments table
CREATE TABLE IF NOT EXISTS departments (
    department_id   SERIAL PRIMARY KEY,
    department_name VARCHAR(100) NOT NULL UNIQUE,
    building        VARCHAR(50),
    phone           VARCHAR(20)
);

-- Teachers table
CREATE TABLE IF NOT EXISTS teachers (
    teacher_id    VARCHAR(10) PRIMARY KEY,
    name          VARCHAR(100) NOT NULL,
    email         VARCHAR(100) UNIQUE NOT NULL,
    department_id INTEGER,
    hire_date     DATE DEFAULT CURRENT_DATE,
    salary        DECIMAL(10, 2),
    FOREIGN KEY (department_id) REFERENCES departments(department_id)
);

-- Students table
CREATE TABLE IF NOT EXISTS students (
    student_id      VARCHAR(10) PRIMARY KEY,
    name            VARCHAR(100) NOT NULL,
    age             INTEGER CHECK (age ≥ 16 AND age ≤ 100),
    email           VARCHAR(100) UNIQUE NOT NULL,
    phone           VARCHAR(20),
    major           VARCHAR(50),
    gpa             DECIMAL(3, 2) CHECK (gpa ≥ 0.0 AND gpa ≤ 4.0),
    department_id   INTEGER,
    enrollment_date DATE DEFAULT CURRENT_DATE,
    FOREIGN KEY (department_id) REFERENCES departments(department_id)
);

-- Courses table
CREATE TABLE IF NOT EXISTS courses (
    course_id     VARCHAR(10) PRIMARY KEY,
    course_name   VARCHAR(100) NOT NULL,
    description   TEXT,
    credits       INTEGER NOT NULL CHECK (credits > 0 AND credits ≤ 6),
    department_id INTEGER,
    teacher_id    VARCHAR(10),
```

```sql
    max_students  INTEGER DEFAULT 30,
    FOREIGN KEY (department_id) REFERENCES departments(department_id),
    FOREIGN KEY (teacher_id) REFERENCES teachers(teacher_id));
-- Enrollments tableCREATE TABLE IF NOT EXISTS enrollments (
    enrollment_id   SERIAL PRIMARY KEY,    student_id      VARCHAR(10) NOT NULL,
    course_id       VARCHAR(10) NOT NULL,    enrollment_date DATE DEFAULT CURRENT_DATE,
    grade           CHAR(1) CHECK (grade IN ('A', 'B', 'C', 'D', 'F', 'W')),
    semester        VARCHAR(20),    year            INTEGER,
    FOREIGN KEY (student_id) REFERENCES students(student_id) ON DELETE CASCADE,
    FOREIGN KEY (course_id)  REFERENCES courses(course_id)   ON DELETE CASCADE,
    UNIQUE(student_id, course_id, semester, year));
-- ==============================================-- SAMPLE DATA
-- ==============================================
INSERT INTO departments (department_name, building, phone) VALUES
    ('Computer Science', 'Engineering Building', '555-0101'),
    ('Mathematics',      'Science Hall',         '555-0102'),
    ('Physics',          'Science Hall',         '555-0103'),
    ('Chemistry',        'Chemistry Building',   '555-0104'),
    ('Engineering',      'Engineering Building', '555-0105')ON CONFLICT DO NOTHING;
INSERT INTO teachers (teacher_id, name, email, department_id, salary) VALUES
    ('T001', 'Dr. John Smith',    'jsmith@university.edu',    1, 85000.00),
    ('T002', 'Dr. Sarah Johnson', 'sjohnson@university.edu',  1, 90000.00),
    ('T003', 'Dr. Michael Brown', 'mbrown@university.edu',    2, 82000.00),
    ('T004', 'Dr. Emily Davis',   'edavis@university.edu',    3, 88000.00),
    ('T005', 'Dr. Robert Wilson', 'rwilson@university.edu',   4, 86000.00)
ON CONFLICT DO NOTHING;
INSERT INTO students (student_id, name, age, email, major, gpa, department_id) VALUES
    ('S001', 'Alice Johnson', 20, 'alice@university.edu',   'Computer Science', 3.8, 1),
    ('S002', 'Bob Smith',     22, 'bob@university.edu',     'Mathematics',      3.5, 2),
    ('S003', 'Charlie Brown', 21, 'charlie@university.edu', 'Physics',          3.9, 3),
    ('S004', 'Diana Prince',  20, 'diana@university.edu',   'Computer Science', 3.7, 1),
    ('S005', 'Eve Wilson',    23, 'eve@university.edu',     'Chemistry',        3.6, 4),
    ('S006', 'Frank Castle',  24, 'frank@university.edu',   'Engineering',      3.4, 5),
    ('S007', 'Grace Hopper',  19, 'grace@university.edu',   'Computer Science', 4.0, 1),
    ('S008', 'Henry Ford',    21, 'henry@university.edu',   'Engineering',      3.3, 5)
ON CONFLICT DO NOTHING;
INSERT INTO courses (course_id, course_name, description, credits, department_id, teacher_id)
        VALUES
    ('CS101',   'Introduction to Programming',  'Learn Java programming basics',           3,
        1, 'T001'),
    ('CS201',   'Data Structures',              'Advanced data structures and algorithms',  4,
        1, 'T002'),
    ('CS301',   'Database Systems',             'Relational databases and SQL',             3,
        1, 'T002'),
    ('MATH101', 'Calculus I',                   'Differential calculus',                    4,
        2, 'T003'),
    ('MATH201', 'Linear Algebra',               'Matrices and vector spaces',               3,
        2, 'T003'),
    ('PHYS101', 'Physics I',                    'Mechanics and thermodynamics',             4,
        3, 'T004'),
```

```sql
    ('CHEM101', 'General Chemistry',            'Introduction to chemistry',               3,
        4, 'T005'),
    ('ENG101',  'Engineering Fundamentals',     'Basic engineering principles',            3,
        5, 'T001')
ON CONFLICT DO NOTHING;
INSERT INTO enrollments (student_id, course_id, grade, semester, year) VALUES
    ('S001', 'CS101',   'A',  'Fall',   2024),    ('S001', 'MATH101', 'B',  'Fall',   2024),
    ('S001', 'CS201',   'A',  'Spring', 2025),    ('S002', 'MATH101', 'A',  'Fall',   2024),
    ('S002', 'CS101',   'B',  'Fall',   2024),    ('S003', 'PHYS101', 'A',  'Fall',   2024),
    ('S004', 'CS101',   'A',  'Fall',   2024),    ('S004', 'CS201',   'B',  'Spring', 2025),
    ('S005', 'CHEM101', 'A',  'Fall',   2024),    ('S006', 'ENG101',  'B',  'Fall',   2024),
    ('S007', 'CS101',   'A',  'Fall',   2024),    ('S007', 'CS201',   'A',  'Spring', 2025),
    ('S007', 'CS301',   NULL, 'Spring', 2025)  -- Currently enrolled, no grade yet
ON CONFLICT DO NOTHING;         -- Confirm everything loaded
SELECT s.student_id, s.name, s.major, s.gpa, d.department_nameFROM students s
JOIN departments d ON s.department_id = d.department_idORDER BY s.student_id;
```

Expected result: 8 student rows, each showing their department name.

💻 PART B — Write the Code (File by File)

> 📌 ***Write the files in order — File 1 → 2 → 3 → 4 → 5. Each file depends on the one before it.***

FILE 1 of 5 — `DatabaseConfig.java`

What this file does: Manages all database connections in one central place. Every other file calls `DatabaseConfig.getConnection()` instead of writing its own connection code.

Open `DatabaseConfig.java` and paste the following:

```java
package com.university.jdbc;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

/**
 * DatabaseConfig — central place for all database settings.
 *
 * HOW TO USE in other files:
```

```
*/     public class DatabaseConfig {
    // ── Change PASSWORD to your own PostgreSQL password ──────────────────────
    private static final String URL      = "jdbc:postgresql://localhost:5432/university_db";
    private static final String USERNAME = "postgres";
    private static final String PASSWORD = "your_password"; // ⚠️ CHANGE THIS!
    // ──────────────────────────────────────────────────────────────────────────
    /**      * Opens a new connection to university_db.
     * Call this at the start of every database operation.
     * Throws SQLException if connection fails — the caller must handle it.     */
    public static Connection getConnection() throws SQLException {        try {
            return DriverManager.getConnection(URL, USERNAME, PASSWORD);
        } catch (SQLException e) {
            System.out.println("✗ Cannot connect to database: " + e.getMessage());
            System.out.println("  → Is PostgreSQL running?");
            System.out.println("  → Is the password correct in DatabaseConfig.java?");
            System.out.println("  → Does university_db exist in pgAdmin?");
            throw e;        }    }                /**
     * Safely closes all three JDBC resources in the correct order:
     *   ResultSet → Statement → Connection     *
     * Always call this in your finally block.
     * Pass null for any resource you don't have.     *        * Examples:
     *   DatabaseConfig.closeResources(conn, pstmt, rs);   // all three
     *   DatabaseConfig.closeResources(conn, pstmt, null); // no ResultSet     */
    public static void closeResources(Connection conn, Statement stmt, ResultSet rs) {
        if (rs ≠ null) {            try { rs.close(); }
            catch (SQLException e) { System.out.println("Error closing ResultSet: " +
         e.getMessage()); }
        }        if (stmt ≠ null) {            try { stmt.close(); }
            catch (SQLException e) { System.out.println("Error closing Statement: " +
         e.getMessage()); }
        }        if (conn ≠ null) {            try { conn.close(); }
            catch (SQLException e) { System.out.println("Error closing Connection: " +
         e.getMessage()); }
        }    }                /**
     * Run this main() to verify your setup before writing any other code.
     * Expected output: "✓ Connection successful! university_db is ready."     */
    public static void main(String[] args) {
        System.out.println("Testing database connection to university_db...");
        Connection conn = null;        try {            conn = getConnection();
            System.out.println("✓ Connection successful! university_db is ready.");
        } catch (SQLException e) {
            System.out.println("✗ Connection failed. Fix the error above before
         continuing.");
        } finally {            closeResources(conn, null, null);        }    }    }
```

✅ Test File 1 Before Continuing

Right-click anywhere inside `DatabaseConfig.java` → **Run 'DatabaseConfig.main()'**

Expected output:

```
Testing database connection to university_db ...
✓ Connection successful! university_db is ready.
```

❌ **If you see an error, stop and fix it now.** Common causes:
- PostgreSQL service is not running — look for the elephant icon in your system tray
- Wrong password — update `PASSWORD` in `DatabaseConfig.java`
- `university_db` doesn't exist — create it in pgAdmin and run the schema SQL from Step 4

FILE 2 of 5 — `Student.java`

What this file does: Represents one row from the `students` table as a Java object. This is called a **Model class** or **POJO** (Plain Old Java Object). The fields match the **exact columns** in yourWeek 7 `students` table — including `phone`, `departmentId`, and `enrollmentDate`.

```java
package com.university.jdbc;

/**
 * Student - represents one row from theWeek 7 'students' table.
 *
 * Column mapping (Java field → database column):
 *   studentId      → student_id
 *   name           → name
 *   age            → age
 *   email          → email
 *   phone          → phone           (nullable)
 *   major          → major
 *   gpa            → gpa
 *   departmentId   → department_id   (foreign key to departments)
 *   enrollmentDate → enrollment_date (set automatically by PostgreSQL DEFAULT)
 *   departmentName → department_name (comes from JOIN with departments table)
 */
public class Student {

    private String studentId;
    private String name;
    private int    age;
    private String email;
    private String phone;           // nullable - not all students have a phone on file
    private String major;
    private double gpa;
    private int    departmentId;    // foreign key - links to departments table
    private String enrollmentDate;  // stored as String for easy display
    private String departmentName;  // populated from LEFT JOIN with departments
```

```java
    public Student(String studentId, String name, int age, String email,
                   String phone, String major, double gpa, int departmentId) {
        this.studentId    = studentId;        this.name         = name;
        this.age          = age;        this.email        = email;
        this.phone        = phone;        this.major        = major;
        this.gpa          = gpa;        this.departmentId = departmentId;    }
    // ── Constructor for SELECT (all fields including date and joined name) ──
    public Student(String studentId, String name, int age, String email,
                   String phone, String major, double gpa, int departmentId,
                   String enrollmentDate, String departmentName) {
        this.studentId      = studentId;        this.name           = name;
        this.age            = age;        this.email          = email;
        this.phone          = phone;        this.major          = major;
        this.gpa            = gpa;        this.departmentId   = departmentId;
        this.enrollmentDate = enrollmentDate;        this.departmentName = departmentName;
    }
    // ── Getters ──────────────────────────────────────────────────────────
    public String getStudentId()       { return studentId; }
    public String getName()            { return name; }
    public int    getAge()             { return age; }
    public String getEmail()           { return email; }
    public String getPhone()           { return phone; }
    public String getMajor()           { return major; }
    public double getGpa()             { return gpa; }
    public int    getDepartmentId()    { return departmentId; }
    public String getEnrollmentDate()  { return enrollmentDate; }
    public String getDepartmentName()  { return departmentName; }
    // ── Setters ──────────────────────────────────────────────────────────
    public void setStudentId(String studentId)          { this.studentId = studentId; }
    public void setName(String name)                    { this.name = name; }
    public void setAge(int age)                         { this.age = age; }
    public void setEmail(String email)                  { this.email = email; }
    public void setPhone(String phone)                  { this.phone = phone; }
    public void setMajor(String major)                  { this.major = major; }
    public void setGpa(double gpa)                      { this.gpa = gpa; }
    public void setDepartmentId(int departmentId)       { this.departmentId = departmentId; }
    public void setEnrollmentDate(String enrollmentDate){ this.enrollmentDate =
        enrollmentDate; }
    public void setDepartmentName(String departmentName){ this.departmentName =
        departmentName; }
            // ── toString ─────────────────────────────────────────────────────────
    @Override    public String toString() {        return String.format(
            "Student[ID=%-6s  Name=%-15s  Age=%-3d  Major=%-20s  GPA=%.2f  Dept=%s]",
            studentId, name, age, major, gpa,
            (departmentName ≠ null ? departmentName : "dept_id=" + departmentId)        );
    }    }
```

> *No test needed here — this is a data class with no `main` method. The other files use it automatically.*

FILE 3 of 5 — `StudentDAO.java`

What this file does: Defines the **interface** — a contract that lists every operation our system can do with students. It says *what* operations exist, but not *how* they work (that's `StudentDAOImpl`'s job).

```java
package com.university.jdbc;

import java.util.List;

/**
 * StudentDAO - interface (contract) for all student database operations.
 *
 * Why use an interface?
 *   - Clearly documents every operation the system can perform
 *   - Forces a clean structure on StudentDAOImpl
 *   - Makes testing easier - you can swap out the implementation
 *   - This is the DAO (Data Access Object) design pattern
 */
public interface StudentDAO {

    // ── CREATE ──────────────────────────────────────────────────────────

    /** Add a new student row. Returns true if successful. */
    boolean addStudent(Student student);

    // ── READ ────────────────────────────────────────────────────────────

    /** Find one student by ID. Includes department name via JOIN. Returns null if not found.
     */
    Student getStudentById(String studentId);

    /** Get all students with their department names, sorted by name. */
    List<Student> getAllStudents();

    /** Get all students in a specific major, sorted by GPA highest first. */
    List<Student> getStudentsByMajor(String major);

    /** Get all students belonging to a department, sorted by name. */
    List<Student> getStudentsByDepartment(int departmentId);

    /** Get all students with GPA ≥ minGpa (honor roll), sorted by GPA highest first. */
    List<Student> getHonorRollStudents(double minGpa);

    // ── UPDATE ──────────────────────────────────────────────────────────

    /** Update all editable fields of a student. Returns true if successful. */
    boolean updateStudent(Student student);

    /** Update only the GPA for a student. Returns true if successful. */
```

```
    boolean updateGPA(String studentId, double newGpa);
    // ── DELETE ──────────────────────────────────────────────────────────
    /** Delete a student by ID (cascades to enrollments). Returns true if successful. */
    boolean deleteStudent(String studentId);
    // ── STATISTICS ──────────────────────────────────────────────────────
    /** Count total number of students. */    int getStudentCount();
    /** Calculate the average GPA across all students. */    double getAverageGPA();}
```

> *No test needed — just the contract. The next file implements all of these methods.*

FILE 4 of 5 — `StudentDAOImpl.java`

What this file does: The most important file — it contains ALL the SQL. Every method follows the same pattern: get connection → prepare SQL → execute → process result → close resources in `finally`.

Notice that `SELECT` queries use a `LEFT JOIN` **with** `departments` so we can display the department name alongside each student — just like you practised inWeek 7.

```
package com.university.jdbc;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;

/**
 * StudentDAOImpl - complete SQL implementation of StudentDAO.
 *
 * PATTERN used in every method:
 *   1. Declare Connection, PreparedStatement, ResultSet = null
 *   2. conn = DatabaseConfig.getConnection()
 *   3. Prepare and execute the SQL
 *   4. Process the result
 *   5. ALWAYS close resources in the finally block
 *
 * All SELECT queries JOIN with departments to include department_name.
 */
public class StudentDAOImpl implements StudentDAO {

    // ═══════════════════════════════════════════════════════════════════
    //  CREATE
    // ═══════════════════════════════════════════════════════════════════

    /**
     * INSERT a new student row into the students table.
     *
     * SQL: INSERT INTO students
```

```
 *          (student_id, name, age, email, phone, major, gpa, department_id)
 *      VALUES (?, ?, ?, ?, ?, ?, ?, ?)     *
 * enrollment_date is NOT included — PostgreSQL sets it automatically
 * via DEFAULT CURRENT_DATE defined in theWeek 7 schema.     */     @Override
public boolean addStudent(Student student) {        Connection conn = null;
    PreparedStatement pstmt = null;                 try {
        conn = DatabaseConfig.getConnection();
        String sql = "INSERT INTO students " +
                     "(student_id, name, age, email, phone, major, gpa, department_id) " +
                     "VALUES (?, ?, ?, ?, ?, ?, ?, ?)";
        pstmt = conn.prepareStatement(sql);
        // Set each ? in order — index starts at 1, not 0!
        pstmt.setString(1, student.getStudentId());
        pstmt.setString(2, student.getName());
        pstmt.setInt(3,    student.getAge());
        pstmt.setString(4, student.getEmail());
        pstmt.setString(5, student.getPhone());      // nullable — OK to be null
        pstmt.setString(6, student.getMajor());
        pstmt.setDouble(7, student.getGpa());
        pstmt.setInt(8,    student.getDepartmentId());
        // executeUpdate() returns number of rows inserted
        int rowsInserted = pstmt.executeUpdate();            return rowsInserted > 0;
            } catch (SQLException e) {
        if (e.getMessage().contains("duplicate key")) {
            System.out.println("  ✗ That Student ID already exists.");
        } else if (e.getMessage().contains("violates foreign key")) {
            System.out.println("  ✗ Department ID does not exist in the departments
    table.");
        } else if (e.getMessage().contains("check constraint")) {
            System.out.println("  ✗ Invalid data: age must be 16-100, GPA must be 0.0-
    4.0.");
        } else {
            System.out.println("  ✗ Database error: " + e.getMessage());            }
        return false;                 } finally {
        DatabaseConfig.closeResources(conn, pstmt, null);        }    }
// ═══════════════════════════════════════════════════════════════════    //  READ
// ═══════════════════════════════════════════════════════════════════
/**      * SELECT one student by student_id.
 * LEFT JOINs with departments to include department_name.
 * Returns a Student object, or null if not found.     *
 * SQL: SELECT s.*, d.department_name     *      FROM students s
 *      LEFT JOIN departments d ON s.department_id = d.department_id
 *      WHERE s.student_id = ?     *
 * LEFT JOIN is used so students with no department assigned still appear.     */
@Override    public Student getStudentById(String studentId) {
    Connection conn = null;        PreparedStatement pstmt = null;
    ResultSet rs = null;                 try {
        conn = DatabaseConfig.getConnection();
        String sql = "SELECT s.*, d.department_name " +
                     "FROM students s " +
                     "LEFT JOIN departments d ON s.department_id = d.department_id " +
```

```
                        "WHERE s.student_id = ?";
        pstmt = conn.prepareStatement(sql);            pstmt.setString(1, studentId);
                rs = pstmt.executeQuery();    // executeQuery() for SELECT only
        if (rs.next()) {
            return extractStudent(rs);  // convert the row into a Student object
        }            return null;  // student not found
    } catch (SQLException e) {
        System.out.println("  ✗ Error finding student: " + e.getMessage());
        return null;        } finally {
        DatabaseConfig.closeResources(conn, pstmt, rs);        }    }                /**
 * SELECT all students with their department names, sorted alphabetically.
 * Returns an empty List (never null) if no students exist.     *
 * SQL: SELECT s.*, d.department_name     *      FROM students s
 *      LEFT JOIN departments d ON s.department_id = d.department_id
 *      ORDER BY s.name     */     @Override    public List<Student> getAllStudents() {
    List<Student> students = new ArrayList<>();        Connection conn = null;
    PreparedStatement pstmt = null;        ResultSet rs = null;                try {
        conn = DatabaseConfig.getConnection();
        String sql = "SELECT s.*, d.department_name " +
                     "FROM students s " +
                     "LEFT JOIN departments d ON s.department_id = d.department_id " +
                     "ORDER BY s.name";
        pstmt = conn.prepareStatement(sql);            rs = pstmt.executeQuery();
        // rs.next() moves to the next row - returns false when all rows are read
        while (rs.next()) {                students.add(extractStudent(rs));            }
            } catch (SQLException e) {
        System.out.println("  ✗ Error loading students: " + e.getMessage());
    } finally {            DatabaseConfig.closeResources(conn, pstmt, rs);        }
            return students;  // empty list, not null - safer to work with    }
         /**      * SELECT students with a specific major, highest GPA first.     *
 * SQL: SELECT s.*, d.department_name     *      FROM students s
 *      LEFT JOIN departments d ON s.department_id = d.department_id
 *      WHERE s.major = ?     *      ORDER BY s.gpa DESC     */     @Override
public List<Student> getStudentsByMajor(String major) {
    List<Student> students = new ArrayList<>();        Connection conn = null;
    PreparedStatement pstmt = null;        ResultSet rs = null;                try {
        conn = DatabaseConfig.getConnection();
        String sql = "SELECT s.*, d.department_name " +
                     "FROM students s " +
                     "LEFT JOIN departments d ON s.department_id = d.department_id " +
                     "WHERE s.major = ? " +                        "ORDER BY s.gpa DESC";
                pstmt = conn.prepareStatement(sql);            pstmt.setString(1, major);
        rs = pstmt.executeQuery();                    while (rs.next()) {
            students.add(extractStudent(rs));            }
    } catch (SQLException e) {
        System.out.println("  ✗ Error finding students by major: " + e.getMessage());
    } finally {            DatabaseConfig.closeResources(conn, pstmt, rs);        }
            return students;    }                /**
 * SELECT all students belonging to a department, sorted by name.
 * Uses the department_id foreign key from theWeek 7 students table.     *
```

```
 * SQL: SELECT s.*, d.department_name     *      FROM students s
 *      LEFT JOIN departments d ON s.department_id = d.department_id
 *      WHERE s.department_id = ?     *      ORDER BY s.name     */     @Override
public List<Student> getStudentsByDepartment(int departmentId) {
    List<Student> students = new ArrayList<>();        Connection conn = null;
    PreparedStatement pstmt = null;        ResultSet rs = null;                try {
        conn = DatabaseConfig.getConnection();
        String sql = "SELECT s.*, d.department_name " +
                     "FROM students s " +
                     "LEFT JOIN departments d ON s.department_id = d.department_id " +
                     "WHERE s.department_id = ? " +
                     "ORDER BY s.name";
        pstmt = conn.prepareStatement(sql);            pstmt.setInt(1, departmentId);
        rs = pstmt.executeQuery();                    while (rs.next()) {
            students.add(extractStudent(rs));            }
    } catch (SQLException e) {
        System.out.println("  ✗ Error finding students by department: " +
     e.getMessage());
    } finally {            DatabaseConfig.closeResources(conn, pstmt, rs);        }
            return students;    }                /**
 * SELECT students whose GPA ≥ minGpa, highest first. Used for Honor Roll.     *
 * SQL: SELECT s.*, d.department_name     *      FROM students s
 *      LEFT JOIN departments d ON s.department_id = d.department_id
 *      WHERE s.gpa ≥ ?     *      ORDER BY s.gpa DESC     */     @Override
public List<Student> getHonorRollStudents(double minGpa) {
    List<Student> students = new ArrayList<>();        Connection conn = null;
    PreparedStatement pstmt = null;        ResultSet rs = null;                try {
        conn = DatabaseConfig.getConnection();
        String sql = "SELECT s.*, d.department_name " +
                     "FROM students s " +
                     "LEFT JOIN departments d ON s.department_id = d.department_id " +
                     "WHERE s.gpa ≥ ? " +                          "ORDER BY s.gpa DESC";
                pstmt = conn.prepareStatement(sql);
        pstmt.setDouble(1, minGpa);            rs = pstmt.executeQuery();
        while (rs.next()) {                students.add(extractStudent(rs));            }
            } catch (SQLException e) {
        System.out.println("  ✗ Error loading honor roll: " + e.getMessage());
    } finally {            DatabaseConfig.closeResources(conn, pstmt, rs);        }
            return students;    }
// ============================================================================    //  UPDATE
// ============================================================================
/**      * UPDATE all editable fields for an existing student.     *
 * SQL: UPDATE students
 *      SET name=?, age=?, email=?, phone=?, major=?, gpa=?, department_id=?
 *      WHERE student_id=?     *
 * ⚠️ The WHERE clause is CRITICAL — without it every row in the table
 *    would be updated at once!     *
 * Parameter order: SET values first (1-7), then WHERE value (8).     */     @Override
public boolean updateStudent(Student student) {        Connection conn = null;
    PreparedStatement pstmt = null;                try {
```

```
        conn = DatabaseConfig.getConnection();
        String sql = "UPDATE students " +
                     "SET name = ?, age = ?, email = ?, phone = ?, " +
                     "    major = ?, gpa = ?, department_id = ? " +
                     "WHERE student_id = ?";
        pstmt = conn.prepareStatement(sql);
        pstmt.setString(1, student.getName());
        pstmt.setInt(2,    student.getAge());
        pstmt.setString(3, student.getEmail());
        pstmt.setString(4, student.getPhone());
        pstmt.setString(5, student.getMajor());
        pstmt.setDouble(6, student.getGpa());
        pstmt.setInt(7,    student.getDepartmentId());
        pstmt.setString(8, student.getStudentId());  // WHERE - always the last parameter
                return pstmt.executeUpdate() > 0;
    } catch (SQLException e) {
        System.out.println("  ✗ Error updating student: " + e.getMessage());
        return false;        } finally {
        DatabaseConfig.closeResources(conn, pstmt, null);        }    }
/**      * UPDATE only the GPA column for one student.     *
 * SQL: UPDATE students SET gpa = ? WHERE student_id = ?     */     @Override
public boolean updateGPA(String studentId, double newGpa) {        Connection conn = null;
    PreparedStatement pstmt = null;                 try {
        conn = DatabaseConfig.getConnection();
        String sql = "UPDATE students SET gpa = ? WHERE student_id = ?";
        pstmt = conn.prepareStatement(sql);            pstmt.setDouble(1, newGpa);
        pstmt.setString(2, studentId);
        return pstmt.executeUpdate() > 0;                } catch (SQLException e) {
        System.out.println("  ✗ Error updating GPA: " + e.getMessage());
        return false;        } finally {
        DatabaseConfig.closeResources(conn, pstmt, null);        }    }
// ═══════════════════════════════════════════════════════════════    //  DELETE
// ═══════════════════════════════════════════════════════════
/**      * DELETE one student row by student_id.     *
 * SQL: DELETE FROM students WHERE student_id = ?     *
 * ⚠️ Always use WHERE - "DELETE FROM students" deletes EVERYONE!     *
 * TheWeek 7 enrollments table has ON DELETE CASCADE on student_id,
 * so deleting a student also removes their enrollment records automatically.     */
@Override    public boolean deleteStudent(String studentId) {
    Connection conn = null;        PreparedStatement pstmt = null;                try {
        conn = DatabaseConfig.getConnection();
        String sql = "DELETE FROM students WHERE student_id = ?";
        pstmt = conn.prepareStatement(sql);            pstmt.setString(1, studentId);
                return pstmt.executeUpdate() > 0;
    } catch (SQLException e) {
        System.out.println("  ✗ Error deleting student: " + e.getMessage());
        return false;        } finally {
        DatabaseConfig.closeResources(conn, pstmt, null);        }    }
// ═══════════════════════════════════════════════════════════
//  STATISTICS
```

```
    // ═══════════════════════════════════════════════════════════════════════
    /**      * COUNT(*) total number of students.     *
     * SQL: SELECT COUNT(*) AS count FROM students     */     @Override
    public int getStudentCount() {        Connection conn = null;
        PreparedStatement pstmt = null;        ResultSet rs = null;                try {
            conn = DatabaseConfig.getConnection();
            String sql = "SELECT COUNT(*) AS count FROM students";
            pstmt = conn.prepareStatement(sql);            rs = pstmt.executeQuery();
            if (rs.next()) {                return rs.getInt("count");            }
        } catch (SQLException e) {
            System.out.println("  ✗ Error counting students: " + e.getMessage());
        } finally {            DatabaseConfig.closeResources(conn, pstmt, rs);        }
                return 0;    }                /**
     * Calculate average GPA (ignoring NULL values).     *
     * SQL: SELECT AVG(gpa) AS avg_gpa FROM students WHERE gpa IS NOT NULL     */
    @Override    public double getAverageGPA() {        Connection conn = null;
        PreparedStatement pstmt = null;        ResultSet rs = null;                try {
            conn = DatabaseConfig.getConnection();
            String sql = "SELECT AVG(gpa) AS avg_gpa FROM students WHERE gpa IS NOT NULL";
            pstmt = conn.prepareStatement(sql);            rs = pstmt.executeQuery();
            if (rs.next()) {                return rs.getDouble("avg_gpa");            }
                } catch (SQLException e) {
            System.out.println("  ✗ Error calculating average GPA: " + e.getMessage());
        } finally {            DatabaseConfig.closeResources(conn, pstmt, rs);        }
                return 0.0;    }
    // ═══════════════════════════════════════════════════════════════════════
    //  PRIVATE HELPER
    // ═══════════════════════════════════════════════════════════════════════
    /**      * Converts the current ResultSet row into a Student object.     *
     * Called by every read method above — centralising this logic means if a
     * column name changes, we only fix it here, not in five different methods.     *
     * Reads enrollment_date as a String for simple display.
     * Reads department_name from the LEFT JOIN result.     */
    private Student extractStudent(ResultSet rs) throws SQLException {
        // enrollment_date may be null for records inserted without a date
        String enrollmentDate = "";        if (rs.getDate("enrollment_date") != null) {
            enrollmentDate = rs.getDate("enrollment_date").toString();        }
        return new Student(            rs.getString("student_id"),
            rs.getString("name"),            rs.getInt("age"),
            rs.getString("email"),
            rs.getString("phone"),            // nullable column
            rs.getString("major"),            rs.getDouble("gpa"),
            rs.getInt("department_id"),            enrollmentDate,
            rs.getString("department_name")  // from the LEFT JOIN        );    }    }
```

✅ Quick Test for File 4

Add this temporary `main` inside `StudentDAOImpl` (before the last `}`), run it, then **delete it**:

```java
    // TEMPORARY TEST - delete after confirming it works!
    public static void main(String[] args) {
        StudentDAOImpl dao = new StudentDAOImpl();

        System.out.println("═══ DAO Test ═══");
        System.out.println("Total students: " + dao.getStudentCount());
        System.out.printf("Average GPA:    %.2f%n%n", dao.getAverageGPA());

        System.out.println("All students (with department name from JOIN):");
        for (Student s : dao.getAllStudents()) {
            System.out.println("  " + s);
        }

        System.out.println("\nFind S007 (Grace Hopper):");
        System.out.println("  " + dao.getStudentById("S007"));

        System.out.println("\nComputer Science students (highest GPA first):");
        for (Student s : dao.getStudentsByMajor("Computer Science")) {
            System.out.printf("  %-15s  GPA: %.2f  Dept: %s%n",
                s.getName(), s.getGpa(), s.getDepartmentName());
        }

        System.out.println("\nDepartment 1 students:");
        for (Student s : dao.getStudentsByDepartment(1)) {
            System.out.printf("  %-15s  Major: %s%n", s.getName(), s.getMajor());
        }
    }
```

Expected output:

```
═══ DAO Test ═══
Total students: 8
Average GPA:    3.65

All students (with department name from JOIN):
  Student[ID=S001    Name=Alice Johnson    Age=20  Major=Computer Science  GPA=3.80
Dept=Computer Science]
  Student[ID=S002    Name=Bob Smith        Age=22  Major=Mathematics       GPA=3.50
Dept=Mathematics]
   ...

Find S007 (Grace Hopper):
  Student[ID=S007    Name=Grace Hopper     Age=19  Major=Computer Science  GPA=4.00
Dept=Computer Science]

Computer Science students (highest GPA first):
  Grace Hopper     GPA: 4.00  Dept: Computer Science
  Alice Johnson    GPA: 3.80  Dept: Computer Science
```

```
Department 1 students:
  Alice Johnson    Major: Computer Science
  Diana Prince     Major: Computer Science
  Grace Hopper     Major: Computer Science
```

FILE 5 of 5 — `StudentManagementApp.java`

What this file does: The main application — shows the menu, reads user input, calls the DAO, and displays results. This file contains **zero SQL** — all database work is delegated to `StudentDAOImpl`.

```java
package com.university.jdbc;

import java.util.List;
import java.util.Scanner;

/**
 * StudentManagementApp - the console menu application.
 *
 * Responsibilities of this file:
 *   ✅ Display the menu
 *   ✅ Read user input from the keyboard
 *   ✅ Call StudentDAO methods
 *   ✅ Display results to the user
 *
 *   ❌ No SQL code here - all database work is in StudentDAOImpl
 *
 * This separation is the DAO pattern in action.
 */
public class StudentManagementApp {

    private StudentDAO studentDAO;
    private Scanner    scanner;

    public StudentManagementApp() {
        this.studentDAO = new StudentDAOImpl();
        this.scanner    = new Scanner(System.in);
    }

    // ═══════════════════════════════════════════════════════════════════
    //  MENU
    // ═══════════════════════════════════════════════════════════════════

    public void displayMenu() {
        System.out.println();
        System.out.println("╔═══════════════════════════════════════════╗");
        System.out.println("╠    STUDENT MANAGEMENT SYSTEM  (JDBC)      ╣");
        System.out.println("╚═══════════════════════════════════════════╝");
```

```
        System.out.println("  1.  Add New Student                          ");
        System.out.println("  2.  View All Students                        ");
        System.out.println("  3.  Find Student by ID                       ");
        System.out.println("  4.  Find Students by Major                   ");
        System.out.println("  5.  Find Students by Department              ");
        System.out.println("  6.  View Honor Roll  (GPA ≥ 3.5)             ");
        System.out.println("  7.  Update Student GPA                       ");
        System.out.println("  8.  Update Student Information               ");
        System.out.println("  9.  Delete Student                           ");
        System.out.println("  10. View Statistics                          ");
        System.out.println("  11. Exit                                     ");
        System.out.println("                                               ");
        System.out.print("  Enter your choice (1-11): ");    }
    // ═══════════════════════════════════════════════════════════════════════
    //  OPTION 1 — Add New Student
    // ═══════════════════════════════════════════════════════════════════════
    public void addStudent() {
        System.out.println("\n── Add New Student ──────────────────────────");
        try {            System.out.print("  Student ID   (e.g. S009) : ");
            String id = scanner.nextLine().trim();
            System.out.print("  Full Name                : ");
            String name = scanner.nextLine().trim();
            System.out.print("  Age                      : ");
            int age = Integer.parseInt(scanner.nextLine().trim());
            System.out.print("  Email                    : ");
            String email = scanner.nextLine().trim();
            System.out.print("  Phone (press Enter to skip): ");
            String phone = scanner.nextLine().trim();
            if (phone.isEmpty()) phone = null;  // phone is nullable in schema
            System.out.print("  Major                    : ");
            String major = scanner.nextLine().trim();
            System.out.print("  GPA (0.0-4.0)            : ");
            double gpa = Double.parseDouble(scanner.nextLine().trim());
            System.out.println("  Departments: 1=Computer Science  2=Mathematics");
            System.out.println("               3=Physics  4=Chemistry  5=Engineering");
            System.out.print("  Department ID (1-5)      : ");
            int deptId = Integer.parseInt(scanner.nextLine().trim());
            // Validate before hitting the database
            if (id.isEmpty() || name.isEmpty() || email.isEmpty()) {
                System.out.println("  ✗ Student ID, Name, and Email cannot be empty.");
                return;            }            if (gpa < 0.0 || gpa > 4.0) {
                System.out.println("  ✗ GPA must be between 0.0 and 4.0.");
                return;            }            if (age < 16 || age > 100) {
                System.out.println("  ✗ Age must be between 16 and 100 (Week 6
    constraint).");
                return;            }            if (deptId < 1 || deptId > 5) {
                System.out.println("  ✗ Department ID must be 1 to 5.");
                return;            }
            // Use the 8-field constructor — enrollment_date set by PostgreSQL DEFAULT
            Student student = new Student(id, name, age, email, phone, major, gpa, deptId);
```

```java
                    if (studentDAO.addStudent(student)) {
                System.out.println("  ✓ Student added successfully!");            } else {
                System.out.println("  ✗ Failed to add student. See error above.");
            }                 } catch (NumberFormatException e) {
            System.out.println("  ✗ Invalid number entered — please try again.");        }
    }
    // ═══════════════════════════════════════════════════════════════════════
    //  OPTION 2 — View All Students
    // ═══════════════════════════════════════════════════════════════════════
    public void viewAllStudents() {
        System.out.println("\n── All Students ──────────────────────────────");
        List<Student> students = studentDAO.getAllStudents();
        if (students.isEmpty()) {            System.out.println("  No students found.");
            return;        }
        System.out.printf("  %-6s  %-15s  %-4s  %-25s  %-20s  %-4s  %s%n",
            "ID", "Name", "Age", "Email", "Major", "GPA", "Department");
        System.out.println("  " + "─".repeat(105));
        for (Student s : students) {
            System.out.printf("  %-6s  %-15s  %-4d  %-25s  %-20s  %.2f  %s%n",
                s.getStudentId(), s.getName(), s.getAge(), s.getEmail(),
                s.getMajor(), s.getGpa(),
                (s.getDepartmentName() ≠ null ? s.getDepartmentName() : "-"));        }
                 System.out.println("  " + "─".repeat(105));
        System.out.println("  Total: " + students.size() + " student(s)");    }
    // ═══════════════════════════════════════════════════════════════════════
    //  OPTION 3 — Find by ID
    // ═══════════════════════════════════════════════════════════════════════
    public void findStudentById() {
        System.out.println("\n── Find Student by ID ────────────────────────");
        System.out.print("  Enter Student ID: ");
        String id = scanner.nextLine().trim();
        Student student = studentDAO.getStudentById(id);
        if (student ≠ null) {            System.out.println("  ✓ Student found:");
            printStudentCard(student);        } else {
            System.out.println("  ✗ No student found with ID: " + id);        }    }
             // ═══════════════════════════════════════════════════════════════════
    //  OPTION 4 — Find by Major
    // ═══════════════════════════════════════════════════════════════════════
    public void findStudentsByMajor() {
        System.out.println("\n── Find Students by Major ────────────────────");
        System.out.print("  Enter Major (e.g. Computer Science): ");
        String major = scanner.nextLine().trim();
        List<Student> students = studentDAO.getStudentsByMajor(major);
        if (students.isEmpty()) {
            System.out.println("  No students found in major: " + major);        } else {
            System.out.println("  Students in " + major + " (highest GPA first):");
            for (int i = 0; i < students.size(); i++) {
                Student s = students.get(i);
                System.out.printf("  %d. %-15s  GPA: %.2f  Dept: %s%n",
                    i + 1, s.getName(), s.getGpa(),
```

```java
                (s.getDepartmentName() ≠ null ? s.getDepartmentName() : "-"));
        }            System.out.println("  Total: " + students.size() + " student(s)");
    }    }
// ═══════════════════════════════════════════════════════════════════════════
//  OPTION 5 — Find by Department
// ═══════════════════════════════════════════════════════════════════════════
public void findStudentsByDepartment() {
    System.out.println("\n── Find Students by Department ──────────────");
    System.out.println("  Departments: 1=Computer Science  2=Mathematics");
    System.out.println("               3=Physics  4=Chemistry  5=Engineering");
    System.out.print("  Enter Department ID (1-5): ");                 try {
        int deptId = Integer.parseInt(scanner.nextLine().trim());
        List<Student> students = studentDAO.getStudentsByDepartment(deptId);
        if (students.isEmpty()) {
            System.out.println("  No students found in department ID: " + deptId);
        } else {                String deptName = students.get(0).getDepartmentName();
            System.out.println("  Students in " + deptName + ":");
            for (int i = 0; i < students.size(); i++) {
                Student s = students.get(i);
                System.out.printf("  %d. %-15s  Major: %-20s  GPA: %.2f%n",
                    i + 1, s.getName(), s.getMajor(), s.getGpa());                }
            System.out.println("  Total: " + students.size() + " student(s)");
        }        } catch (NumberFormatException e) {
        System.out.println("  ✗ Please enter a number (1-5).");        }    }
// ═══════════════════════════════════════════════════════════════════════════
//  OPTION 6 — Honor Roll
// ═══════════════════════════════════════════════════════════════════════════
public void viewHonorRoll() {
    System.out.println("\n── Honor Roll  (GPA ≥ 3.5) ────────────────");
    List<Student> students = studentDAO.getHonorRollStudents(3.5);
    if (students.isEmpty()) {
        System.out.println("  No students currently on the honor roll.");        } else {
        System.out.printf("  %-4s  %-15s  %-20s  %-22s  %s%n",
            "Rank", "Name", "Major", "Department", "GPA");
        System.out.println("  " + "─".repeat(75));
        for (int i = 0; i < students.size(); i++) {
            Student s = students.get(i);
            System.out.printf("  %-4d  %-15s  %-20s  %-22s  %.2f%n",
                i + 1, s.getName(), s.getMajor(),
                (s.getDepartmentName() ≠ null ? s.getDepartmentName() : "-"),
                s.getGpa());            }
        System.out.println("  Total: " + students.size() + " student(s) on honor roll");
    }    }
// ═══════════════════════════════════════════════════════════════════════════
//  OPTION 7 — Update GPA
// ═══════════════════════════════════════════════════════════════════════
public void updateGPA() {
    System.out.println("\n── Update Student GPA ──────────────────────");
    System.out.print("  Enter Student ID: ");
    String id = scanner.nextLine().trim();
```

```
        Student student = studentDAO.getStudentById(id);        if (student == null) {
            System.out.println("  ✗ No student found with ID: " + id);            return;
        }                System.out.println("  Student: " + student.getName());
        System.out.printf("  Current GPA: %.2f%n", student.getGpa());
        System.out.print("  New GPA (0.0-4.0): ");                try {
            double newGpa = Double.parseDouble(scanner.nextLine().trim());
            if (newGpa < 0.0 || newGpa > 4.0) {
                System.out.println("  ✗ GPA must be between 0.0 and 4.0.");
                return;            }
            if (studentDAO.updateGPA(id, newGpa)) {
                System.out.printf("  ✓ GPA updated: %.2f → %.2f%n", student.getGpa(), newGpa);
            } else {                System.out.println("  ✗ Update failed.");            }
                } catch (NumberFormatException e) {
            System.out.println("  ✗ Invalid number - enter a decimal like 3.75");        }
    }
    // ═══════════════════════════════════════════════════════════════════════
    //  OPTION 8 - Update Student Information
    // ═══════════════════════════════════════════════════════════════════════
    public void updateStudentInfo() {
        System.out.println("\n── Update Student Information ────────────────");
        System.out.print("  Enter Student ID: ");
        String id = scanner.nextLine().trim();
        Student student = studentDAO.getStudentById(id);        if (student == null) {
            System.out.println("  ✗ No student found with ID: " + id);            return;
        }                System.out.println("  Current information:");
        printStudentCard(student);
        System.out.println("  Press Enter to keep the current value for any field.\n");
                System.out.print("  Name  [" + student.getName() + "]: ");
        String input = scanner.nextLine().trim();
        if (!input.isEmpty()) student.setName(input);
        System.out.print("  Age   [" + student.getAge() + "]: ");
        input = scanner.nextLine().trim();        if (!input.isEmpty()) {
            try { student.setAge(Integer.parseInt(input)); }
            catch (NumberFormatException e) { System.out.println("  (Invalid age - keeping
         current)"); }
        }                System.out.print("  Email [" + student.getEmail() + "]: ");
        input = scanner.nextLine().trim();
        if (!input.isEmpty()) student.setEmail(input);
        System.out.print("  Phone [" + (student.getPhone() ≠ null ? student.getPhone() : "")
         + "]: ");
        input = scanner.nextLine().trim();
        if (!input.isEmpty()) student.setPhone(input);
        System.out.print("  Major [" + student.getMajor() + "]: ");
        input = scanner.nextLine().trim();
        if (!input.isEmpty()) student.setMajor(input);
        System.out.print("  GPA   [" + student.getGpa() + "]: ");
        input = scanner.nextLine().trim();        if (!input.isEmpty()) {
            try { student.setGpa(Double.parseDouble(input)); }
            catch (NumberFormatException e) { System.out.println("  (Invalid GPA - keeping
         current)"); }
```

```
        }
        System.out.println("  Departments: 1=Computer Science  2=Mathematics  3=Physics
         4=Chemistry  5=Engineering");
        System.out.print("  Dept ID [" + student.getDepartmentId() + "]: ");
        input = scanner.nextLine().trim();        if (!input.isEmpty()) {
            try { student.setDepartmentId(Integer.parseInt(input)); }
            catch (NumberFormatException e) { System.out.println("  (Invalid department —
         keeping current)"); }
        }                 if (studentDAO.updateStudent(student)) {
            System.out.println("  ✓ Student information updated successfully!");
        } else {            System.out.println("  ✗ Update failed.");        }    }
            // ═══════════════════════════════════════════════════════════════════
    //  OPTION 9 — Delete Student
    // ═══════════════════════════════════════════════════════════════════
    public void deleteStudent() {
        System.out.println("\n── Delete Student ──────────────────────────");
        System.out.print("  Enter Student ID: ");
        String id = scanner.nextLine().trim();
        Student student = studentDAO.getStudentById(id);        if (student == null) {
            System.out.println("  ✗ No student found with ID: " + id);            return;
        }                 System.out.println("  You are about to delete:");
        printStudentCard(student);
        System.out.println("  ⚠  Their enrollment records will also be deleted (ON DELETE
         CASCADE).");
        System.out.print("  Type 'yes' to confirm: ");
        String confirm = scanner.nextLine().trim();
        if (confirm.equalsIgnoreCase("yes")) {            if (studentDAO.deleteStudent(id)) {
                System.out.println("  ✓ Student and their enrollment records deleted.");
            } else {                System.out.println("  ✗ Deletion failed.");            }
        } else {
            System.out.println("  Deletion cancelled — no changes were made.");        }
    }
    // ═══════════════════════════════════════════════════════════════════
    //  OPTION 10 — Statistics
    // ═══════════════════════════════════════════════════════════════════
    public void viewStatistics() {
        System.out.println("\n── System Statistics ───────────────────────");
        int    total      = studentDAO.getStudentCount();
        double avgGpa     = studentDAO.getAverageGPA();
        int    honorCount = studentDAO.getHonorRollStudents(3.5).size();
        System.out.println("  Total students      : " + total);
        System.out.printf("  Average GPA         : %.2f%n", avgGpa);
        System.out.println("  On honor roll       : " + honorCount + "  (GPA ≥ 3.5)");
                if (total > 0) {
            System.out.printf("  Honor roll %%        : %.1f%%%n",
                (honorCount * 100.0) / total);        }    }
    // ═══════════════════════════════════════════════════════════════════
    //  PRIVATE HELPER — formatted student detail card
    // ═══════════════════════════════════════════════════════════════════
    private void printStudentCard(Student s) {
        System.out.println("  ┌──────────────────────────────────────────┐");
```

```
        System.out.printf("   │  ID            : %-27s│%n", s.getStudentId());
        System.out.printf("   │  Name          : %-27s│%n", s.getName());
        System.out.printf("   │  Age           : %-27d│%n", s.getAge());
        System.out.printf("   │  Email         : %-27s│%n", s.getEmail());
        System.out.printf("   │  Phone         : %-27s│%n",
            (s.getPhone() ≠ null ? s.getPhone() : "–"));
        System.out.printf("   │  Major         : %-27s│%n", s.getMajor());
        System.out.printf("   │  GPA           : %-27.2f│%n", s.getGpa());
        System.out.printf("   │  Department    : %-27s│%n",
            (s.getDepartmentName() ≠ null ? s.getDepartmentName()
                                          : "dept_id=" + s.getDepartmentId()));
        System.out.printf("   │  Enrolled      : %-27s│%n",
            (s.getEnrollmentDate() ≠ null && !s.getEnrollmentDate().isEmpty()
                ? s.getEnrollmentDate() : "–"));
        System.out.println("   └──────────────────────────────────────────┘");    }
            // ═══════════════════════════════════════════════════════════════════
    //  RUN – main application loop
    // ═══════════════════════════════════════════════════════════════════
    public void run() {
        System.out.println("╔═══════════════════════════════════════╗");
        System.out.println("║   Welcome to Student Management System   ");
        System.out.println("║     Connected to: university_db (JDBC)   ");
        System.out.println("╚═══════════════════════════════════════╝");
        // Verify the database is reachable before showing the menu        try {
            DatabaseConfig.getConnection().close();
            System.out.println("✓ Connected to university_db successfully!\n");
        } catch (Exception e) {
            System.out.println("✗ Cannot connect to database!");
            System.out.println("  Fix DatabaseConfig.java then restart.\n");
            return;        }                int choice;        do {            displayMenu();
            try {                choice = Integer.parseInt(scanner.nextLine().trim());
                switch (choice) {
                    case 1:  addStudent();               break;
                    case 2:  viewAllStudents();          break;
                    case 3:  findStudentById();          break;
                    case 4:  findStudentsByMajor();      break;
                    case 5:  findStudentsByDepartment(); break;
                    case 6:  viewHonorRoll();            break;
                    case 7:  updateGPA();                break;
                    case 8:  updateStudentInfo();        break;
                    case 9:  deleteStudent();            break;
                    case 10: viewStatistics();           break;                    case 11:
                        System.out.println("\n  Goodbye! Connection closed.");
                        break;                    default:
                        System.out.println("  ✗ Enter a number from 1 to 11.");
                }            } catch (NumberFormatException e) {
                System.out.println("  ✗ Invalid input – please type a number.");
                choice = 0;            }        } while (choice ≠ 11);
        scanner.close();    }
    // ═══════════════════════════════════════════════════════════════════
```

```java
    //  MAIN - entry point for the whole application
    // ==========================================================================
    public static void main(String[] args) {        new StudentManagementApp().run();    }
}
```

🏃 PART C — Run the Application

Step 1 — Launch

Right-click `StudentManagementApp.java` → **Run 'StudentManagementApp.main()'**

```
╔═══════════════════════════════════════════╗
   Welcome to Student Management System
     Connected to: university_db (JDBC)
╚═══════════════════════════════════════════╝

✓ Connected to university_db successfully!

╔═══════════════════════════════════════════╗
    STUDENT MANAGEMENT SYSTEM  (JDBC)
╠═══════════════════════════════════════════╣
  1.  Add New Student
...
  Enter your choice (1-11):
```

Step 2 — Test Every Menu Option

Work through this checklist in order to confirm everything works:

#	Test	Option	What to enter	Expected result
1	View all	2	—	8 students with department names from JOIN
2	Find by ID	3	`S001`	Alice Johnson's full card
3	Find by ID	3	`S007`	Grace Hopper — GPA 4.0, Computer Science

4	Find by major	4	`Computer Science`	Grace, Alice, Diana — sorted GPA high→low
5	Find by dept	5	`1`	3 Computer Science students
6	Find by dept	5	`5`	Frank Castle and Henry Ford (Engineering)
7	Honor roll	6	—	6 students with GPA ≥ 3.5, ranked
8	Add student	1	ID `S009`, name, age `20`, email, major `Mathematics`, GPA `3.7`, dept `2`	"Added successfully!"
9	Find new	3	`S009`	New student card with department name
10	Update GPA	7	`S009`, new GPA `3.9`	"GPA updated: 3.70 → 3.90"
11	Update info	8	`S009`, change phone	"Updated successfully!"
12	Statistics	10	—	Count 9, updated average
13	Delete	9	`S009`, type `yes`	"Student and enrollments deleted"
14	View all again	2	—	Back to 8 students

🔍 PART D — How Everything Connects

```
StudentManagementApp                    (user menu – zero SQL)
│
│  User picks option 5 "Find by Department"
│  → findStudentsByDepartment() reads department ID from keyboard
│  → calls:
│
└──→ studentDAO.getStudentsByDepartment(1)
        │
        │  studentDAO is actually a StudentDAOImpl object
```

```
│
        ├──► DatabaseConfig.getConnection()
        │        └──► DriverManager opens socket to PostgreSQL
        │
        ├──► conn.prepareStatement(sql)
        │        sql = "SELECT s.*, d.department_name
        │               FROM students s
        │               LEFT JOIN departments d
        │                 ON s.department_id = d.department_id
        │               WHERE s.department_id = ?
        │               ORDER BY s.name"
        │
        ├──► pstmt.setInt(1, 1)          ← sets the ? to department_id = 1
        ├──► pstmt.executeQuery()         ← PostgreSQL runs the SQL
        │        └──► ResultSet: 3 rows (Alice, Diana, Grace)
        │
        ├──► while (rs.next()) → extractStudent(rs) × 3
        │        └──► each ResultSet row → Student object → added to List
        │
        └──► DatabaseConfig.closeResources(conn, pstmt, rs)
                 └──► closes ResultSet → Statement → Connection
        │
        returns List<Student> with 3 students
        │
└──► findStudentsByDepartment() prints the list
```

⚠️ Common Mistakes to Avoid

Mistake 1 — Using Statement instead of PreparedStatement (SQL Injection!)

```java
// ❌ WRONG – attacker types: ' OR '1'='1 and bypasses your filter
String sql = "SELECT * FROM students WHERE name = '" + userInput + "'";

// ✅ CORRECT – ? is safely escaped, injection is impossible
String sql = "SELECT * FROM students WHERE name = ?";
pstmt.setString(1, userInput);
```

Mistake 2 — Wrong parameter index (most common beginner error!)

```java
// ❌ WRONG – index starts at 1, not 0
pstmt.setString(0, studentId);   // throws SQLException!

// ✅ CORRECT
pstmt.setString(1, studentId);   // first ?  = index 1
```

```
pstmt.setString(2, name);        // second ? = index 2
pstmt.setInt(3,    age);         // third ?  = index 3
```

Mistake 3 — Wrong execute method

```
// ❌ WRONG — executeQuery for a write operation
pstmt.executeQuery();   // only use for SELECT

// ✅ CORRECT
pstmt.executeQuery();    // SELECT → returns ResultSet
pstmt.executeUpdate();   // INSERT / UPDATE / DELETE → returns row count
```

Mistake 4 — Forgetting to close resources

```
// ❌ WRONG — if an exception occurs, conn is never closed!
Connection conn = DatabaseConfig.getConnection();
// ... exception thrown here ...
conn.close();   // never reached → connection leaks!

// ✅ CORRECT — finally always runs
try {
    conn = DatabaseConfig.getConnection();
    // ... do work ...
} catch (SQLException e) {
    System.out.println("Error: " + e.getMessage());
} finally {
    DatabaseConfig.closeResources(conn, pstmt, rs);  // always runs
}
```

Mistake 5 — Missing WHERE clause on UPDATE or DELETE

```
// ❌ DANGER — updates every single student in the database!
"UPDATE students SET gpa = 4.0"

// ❌ DANGER — deletes every student from the database!
"DELETE FROM students"

// ✅ CORRECT — always target one specific row
"UPDATE students SET gpa = ? WHERE student_id = ?"
"DELETE FROM students WHERE student_id = ?"
```

Mistake 6 — Wrong column name in ResultSet (specific toWeek 7 schema)

```
// ❌ WRONG – column is student_id not id, and phone exists inWeek 7
rs.getString("id");            // no such column
rs.getString("dept");          // no such column

// ✅ CORRECT – use the exactWeek 7 column names
rs.getString("student_id");
rs.getString("phone");
rs.getInt("department_id");
rs.getString("department_name");  // only available after LEFT JOIN
rs.getDate("enrollment_date");
```

🏆 Extension Challenges (Homework)

Now that the base system works with your fullWeek 7 schema, try adding these:

Challenge 1 — Search Students by Name (LIKE)

```
// Add to StudentDAO:
List<Student> searchByName(String keyword);

// In StudentDAOImpl – partial match, case-insensitive:
String sql = "SELECT s.*, d.department_name " +
             "FROM students s " +
             "LEFT JOIN departments d ON s.department_id = d.department_id " +
             "WHERE LOWER(s.name) LIKE LOWER(?) " +
             "ORDER BY s.name";
pstmt.setString(1, "%" + keyword + "%");
```

Challenge 2 — View a Student's Enrolled Courses

```
// New method – joins enrollments → courses using yourWeek 7 tables:
String sql = "SELECT c.course_id, c.course_name, c.credits, " +
             "       e.grade, e.semester, e.year " +
             "FROM enrollments e " +
             "JOIN courses c ON e.course_id = c.course_id " +
             "WHERE e.student_id = ? " +
             "ORDER BY e.year DESC, e.semester";
```

Challenge 3 — Department Summary Report

```java
// GROUP BY departments — a realWeek 7 JOIN + aggregate query in Java:
String sql = "SELECT d.department_name, " +
             "       COUNT(s.student_id) AS student_count, " +
             "       ROUND(AVG(s.gpa)::numeric, 2) AS avg_gpa, " +
             "       MAX(s.gpa) AS top_gpa " +
             "FROM departments d " +
             "LEFT JOIN students s ON d.department_id = s.department_id " +
             "GROUP BY d.department_id, d.department_name " +
             "ORDER BY d.department_name";
```

Challenge 4 — Enroll a Student in a Course (Transaction)

```java
// Both steps must succeed together — use a transaction:
conn.setAutoCommit(false);  // start transaction
try {
    // Step 1: check student exists
    // Step 2: check course exists and has capacity
    // Step 3: INSERT into enrollments
    conn.commit();          // save all changes
} catch (SQLException e) {
    conn.rollback();        // undo everything if any step failed
} finally {
    conn.setAutoCommit(true);
}
```

📌 Quick Reference Card

DAO Methods → SQL Mapping

I want to...	Method	SQL pattern
Add a student	`addStudent(student)`	`INSERT INTO students (8 columns) VALUES (?,?,?,?,?,?,?,?)`
Find one student	`getStudentById(id)`	`SELECT s.*, d.name ... WHERE student_id = ?`

Filter by major	`getStudentsByMajor(major)`	`... WHERE major = ? ORDER BY gpa DESC`
Filter by department	`getStudentsByDepartment(id)`	`... WHERE department_id = ? ORDER BY name`
Honor roll	`getHonorRollStudents(3.5)`	`... WHERE gpa ≥ ? ORDER BY gpa DESC`
Change GPA only	`updateGPA(id, gpa)`	`UPDATE students SET gpa = ? WHERE student_id = ?`
Change all fields	`updateStudent(student)`	`UPDATE students SET 7 cols WHERE student_id = ?`
Remove a student	`deleteStudent(id)`	`DELETE FROM students WHERE student_id = ?` (cascades)
Count all	`getStudentCount()`	`SELECT COUNT(*) FROM students`
Average GPA	`getAverageGPA()`	`SELECT AVG(gpa) FROM students WHERE gpa IS NOT NULL`

JDBC Rules — Always Remember

Rule	What to do
Always use	`PreparedStatement` — never `Statement`
Parameter index	Starts at **1**, not 0
For SELECT	`executeQuery()` → returns `ResultSet`
For INSERT / UPDATE / DELETE	`executeUpdate()` → returns row count
Always in finally	`DatabaseConfig.closeResources(conn, pstmt, rs)`
Always add WHERE	to every UPDATE and DELETE
Column names	Must exactly match yourWeek 7 schema
JOIN for dept name	`LEFT JOIN departments d ON s.department_id = d.department_id`

Week 7 Hands-On Guide — Student Management System with JDBC
Built directly on theWeek 7 `university_db` *schema (departments, students, teachers, courses, enrollments)*
Next: Week 8 Mid-Course Project